

前置判断

在正式分析前，先回答一个关键问题：**如果去掉AI，这个产品是否仍然基本成立？**

- 如果去掉AI后，产品仍然可以作为传统软件运行，只是少了一个智能功能，那么它更可能是“AI产品”。
- 如果去掉AI后，产品的核心交互、任务流程、价值主张都无法成立，那么它才更接近“AI Native产品”。

AI产品与AI Native产品的核心区别

维度	AI 产品	AI Native产品
产品逻辑	在原有产品中加入AI能力	围绕AI能力重新设计产品
用户任务链	原任务链基本不变	任务链被压缩、重组或代理化
AI角色	辅助功能、插件、提效模块	任务执行者、决策辅助者、协作主体
交互方式	按钮、表单、菜单为主，AI作为入口之一	自然语言、多轮对话、主动建议、任务编排
上下文需求	局部上下文即可	需要持续、跨系统、可管理的上下文
可靠性要求	AI出错后用户可自行修正	必须设计校验、兜底、回滚、权限和审计
商业价值	提效、降本、增强体验	重构工作流、降低技能门槛、创造新能力

总览

把以下产品分析结论作为文章分析的开头，建议回答五个问题：

1. **该产品改变了什么任务链？**
 - 是压缩步骤、自动执行、降低技能门槛，还是创造新任务等？
2. **AI在其中承担什么角色？**
 - 助手、Copilot、生成者、Agent，还是基础设施等？
3. ****该产品拿到了什么关键上下文？**
 - 上下文是否构成产品壁垒？

4. ****该产品如何处理AI不可靠的问题？**

- 是否有审核、引用、回滚、权限、日志和兜底？

5. **用户为什么愿意付费？**

- 是为了省时间、省人力、提质量，还是获得过去没有的新能力？

1. 产品概览

1.1 基本信息

- 产品名称 / 公司
- 上线时间与发展阶段
- 一句话定位：产品解决了什么核心问题？
- 产品类别：
 - 通用对话助手
 - 办公协作 Copilot
 - 代码协同开发
 - 知识研究工作台
 - 自主执行业务 Agent
 - 其他类型（备注）

1.2 核心价值主张

重点分析：

- 用户为什么需要它？
- 它替代了什么旧工具、旧流程或旧角色？
- 它是提升效率，还是创造了过去难以完成的新任务？
- 它的核心价值来自模型能力，还是来自模型与场景、数据、工具的组合？

1.3 AI Native程度初判

可以用以下问题判断：

1. AI是否位于产品主路径，而不是边缘功能？
 2. 用户是否通过“表达目标”而不是“操作功能”来使用产品？
 3. AI是否参与任务拆解、执行、判断或交付？
 4. 产品是否围绕AI的不确定性设计了可靠性机制？
 5. 用户付费是否因为任务方式被重构，而不仅是功能增强？
-

2. 用户与任务链

AI Native产品的核心分析对象不是功能，而是**任务链**。

2.1 核心用户

- 直接使用者是谁？
- 付费决策者是谁？
- 受影响的协作者是谁？
- 使用者与决策者是否分离？

例如，企业AI产品中，使用者可能是一线员工，付费者可能是部门负责人或CIO，风险承担者可能是合规、法务或IT团队。

2.2 用户目标

回答：

- 用户在什么场景下打开产品？
- 用户真正想完成的任务是什么？
- 用户过去如何完成这个任务？
- 原流程中最痛的环节是什么：信息获取、分析判断、内容生成、系统操作、跨部门协同，还是结果校验？

2.3 AI在任务链中的角色

按实际作用分类：

1. 助手型

- AI回答问题、解释信息、辅助理解。
- 用户仍然主导任务。
- 更接近普通AI产品，AI Native程度较弱。

2. Copilot型

- 人主导操作，AI持续提供建议、补全、生成或局部自动化。
- 典型场景：代码补全、文档写作、设计辅助。

3. 生成决策型

- AI直接生成较完整的交付物。
- 用户主要承担审核、修改和确认。
- 典型场景：报告、方案、邮件、代码、图像、视频生成。

4. Agent型

- 用户设定目标和约束，AI自主拆解任务、调用工具、执行多步操作。
- 人从操作者变成监督者。

- 这是AI Native程度最高的形态之一。

5. Infra/API型

- 产品不直接面向终端用户，而是为其他产品提供AI能力。
- 分析重点从用户体验转向开发者体验、模型能力、调用成本、稳定性和生态集成。

2.4 任务链拆解

将核心任务拆解为不超过7步

步骤	用户任务	传统方式	AI参与方式	自动化程度	人的角色	风险等级
1						
2						
3						
4						
5						
6						
7						

分析重点不是“AI做了多少”，而是：**AI是否进入了任务链的关键节点。**

3. 上下文与数据系统

AI Native产品的能力上限，往往不只取决于模型，而取决于它能拿到什么上下文。

3.1 用户显式输入

包括：

- Prompt
- 文档
- 语音
- 图像
- 视频
- 表格
- 代码
- 用户选择的目标、约束和偏好
- 其他（备注）

分析重点：

- 用户输入成本高不高？
- 产品是否帮助用户澄清需求？
- 是否支持多轮补充？
- 输入是否结构化？

3.2 产品读取的上下文

分为两类：

内部上下文

- 历史会话
- 文件与项目资料
- 用户画像
- **工作空间**
- 团队知识库
- 过往输出结果
- 用户偏好与修改记录
- 其他（备注）

外部上下文

- Web检索
- 邮件
- 日历
- 企业数据库
- CRM / ERP / OA系统
- 代码库
- 第三方API
- 实时市场数据
- 外部文档库
- 其他（备注）

AI Native产品通常不是只依赖一次Prompt，而是依赖持续积累和动态调用的上下文系统。

3.3 上下文处理方式

需要分析：

- 是否做信息抽取？

- 是否做摘要压缩？
- 是否使用向量数据库或RAG？
- 是否区分事实、偏好、任务状态和临时信息？
- 是否有上下文优先级机制？
- 是否避免过期信息污染当前决策？
- 其他（备注）

3.4 记忆策略

重点看：

- 是否有短期记忆：当前任务、当前会话、当前文件。
- 是否有长期记忆：用户偏好、项目背景、组织知识。
- 是否有任务记忆：已完成步骤、待办事项、失败记录。
- 是否有选择性遗忘：用户能否删除、关闭或编辑记忆。
- 是否有记忆可见性：用户是否知道AI记住了什么。

AI Native产品的记忆不是“多存信息”，而是要做到：**该记住的记住，该遗忘的遗忘，该暴露给用户确认的暴露出来。**

3.5 上下文可控性与可见性

重点问题：

- 用户能否查看AI使用了哪些信息？
- 用户能否排除某些信息？
- 用户能否锁定某些事实作为真源？
- AI是否区分“引用事实”和“推测判断”？
- 企业场景下是否严格遵守权限边界？

4. 模型、工具与系统架构

AI Native产品不是“模型套壳”，而是由模型、上下文、工具、 workflows、评估和权限系统共同构成。

4.1 模型依赖

分析：

- 使用自研模型，还是调用外部模型？
- 是否支持多模型路由？

- 是否按任务类型选择不同模型？
- 是否有小模型承担低成本任务，大模型承担复杂推理任务？
- 是否支持本地化或私有化部署？

4.2 模型适配方式

包括：

- Prompt Engineering
- RAG
- SFT微调
- RLHF / RLAIIF
- 工具调用
- workflow编排
- 多Agent协作
- 结构化输出约束
- 其他（备注）

判断重点：

- 产品差异化来自模型本身，还是来自场景适配？
- 是否建立了领域知识、业务规则和工作流约束？
- 输出是否可稳定复现？

4.3 工具调用能力

Agent型产品尤其要分析：

- AI能调用哪些工具？
- 工具调用是否需要用户授权？
- 是否有权限边界？
- 是否支持沙盒执行？
- 是否能感知工具执行结果？
- 工具失败后是否能重试或换路径？

工具包括：

- 搜索
- 数据库查询
- 文件读写
- 邮件发送

- 日程创建
- 代码执行
- API调用
- 系统操作
- 浏览器自动化
- 其他（备注）

4.4 非功能性指标

AI Native产品必须关注工程指标：

- 首Token时间
- 总生成时延
- 流式体验
- 单次任务成本
- 上下文窗口限制
- 高并发稳定性
- 工具调用成功率
- 超时率
- 重试率
- 多轮任务完成率

这些指标会直接影响产品是否可商业化，而不只是技术体验。

5. 数据飞轮与评测体系

AI Native产品的长期壁垒，很大程度来自数据飞轮和评测体系。

5.1 用户反馈回流

分析：

- 用户是否点赞/点踩？
- 用户是否修改AI输出？
- 系统是否记录修改前后差异？
- 是否识别用户采纳了哪些建议？
- 是否记录任务是否完成？
- 是否区分“用户不喜欢”和“事实错误”？

5.2 数据飞轮类型

常见飞轮包括：

1. 使用数据飞轮

- 用户越多，场景越丰富，产品越懂任务。

2. *反馈数据飞轮

- 用户修改越多，系统越能学习偏好和高质量输出模式。

3. 领域数据飞轮

- 产品积累行业知识、模板、规则、案例，形成垂直壁垒。

4. workflow 数据飞轮

- 产品理解任务链条中的步骤、失败点和最佳路径。

• 5. 其他（备注）

5.3 评测体系

需要同时看业务指标和技术指标。

业务指标

- 任务完成率
- 输出采纳率
- 人工修改率
- 节省时间
- 留存率
- 复购率
- 企业部署扩张率
- 单用户任务量

技术指标

- 事实一致性
- 幻觉率
- 引用准确率
- 格式合规率
- 工具调用成功率
- 推理正确率
- 安全违规率
- 多轮任务稳定性

6. 输出生成与可靠性

AI Native产品的核心风险是：它既强大，又不完全可靠。因此可靠性设计是产品的一部分，而不是技术附属项。

6.1 输出类型

包括：

- 文本
- 结构化数据
- 代码
- 图像
- 视频
- 音频
- 操作指令
- API调用结果
- 工作流执行结果
- 决策建议
- 其他（备注）

输出越接近真实世界行动，可靠性要求越高。

6.2 验证机制

分析：

- 是否提供引用来源？
- 是否展示证据链？
- 是否标注置信度？
- 是否支持事实核查？
- 是否有规则校验？
- 是否支持结构化Schema校验？
- 是否提供Diff对比？
- 是否支持版本管理？
- 是否支持Undo、回滚和撤销？

例如，AI写一段营销文案出错，风险较低；AI自动发送重要邮件、修改代码库、调整金融参数，风险就显著提高。

6.3 人机协同节点

需要区分：

- AI可直接生成，但必须人审。
- AI可提出建议，但不能自动执行。
- AI可执行低风险操作。
- AI执行高风险操作前必须二次确认。
- AI永远不能执行某些任务。

AI Native产品成熟度的重要标志，是**能根据任务风险动态设计人工介入点**。

6.4 可靠性分层

可以按风险分为四层：

层级	AI权限	适用场景	产品机制
L1 建议层	只提供建议	问答、解释、草稿	用户自行判断
L2 生成层	生成交付物	文案、代码、报告	人工审核、版本对比
L3 执行层	调用工具执行	发邮件、查数据、改文件	授权、日志、回滚
L4 自主层	多步自主完成任务	Agent工作流	权限隔离、监控、审计、兜底

7. 信任、合规与安全

AI Native产品越深入任务链，越需要建立信任系统。

7.1 可解释性

分析：

- AI是否说明依据？
- 用户能否追问“你为什么这么判断”？
- AI是否区分事实、推理和建议？
- 是否展示引用、来源和工具调用记录？
- 是否避免伪造推理过程？

可解释性不是展示一长串“思考过程”，而是让用户知道：**AI基于什么信息得出什么结论。**

7.2 隐私与数据安全

重点问题：

- 用户数据是否用于训练？
- 企业数据是否进入第三方模型？
- 是否支持数据不出域？
- 是否有加密、脱敏和访问控制？
- 是否符合GDPR、HIPAA、金融监管、等保等要求？
- 是否支持企业管理员管理数据策略？

7.3 权限隔离

尤其在企业产品中，需要分析：

- AI是否只能访问用户有权访问的数据？
- 团队知识库是否有角色级权限？
- Agent调用工具时是否继承用户权限？
- 是否防止越权读取、越权总结、越权执行？

7.4 对抗鲁棒性

包括：

- 防提示注入
- 防越狱
- 防数据泄露
- 防恶意文件诱导
- 防工具调用滥用
- 敏感内容识别
- 高风险操作拦截

7.5 用户终极控制权

用户应拥有：

- 数据删除权
- 记忆关闭权
- 输出撤回权
- 操作确认权

- 安全策略自定义权
- 企业级审计权

AI Native产品不是让AI替代所有控制，而是让用户从低价值操作中解放出来，同时保留关键控制权。

8. 失败兜底与异常处理

AI Native产品必须假设失败一定会发生。

8.1 失败类型

常见失败包括：

1. 模型失败

- 事实错误
- 幻觉
- 逻辑断裂
- 指令不遵守
- 格式错误
- 风格不稳定
- 其他（备注）

2. 上下文失败

- 读取了错误信息
- 遗漏关键上下文
- 使用过期信息
- 权限不足
- RAG召回错误
- 其他（备注）

3. 工具失败

- API调用失败
- 超时
- 权限不足
- 参数错误
- 外部系统不可用
- 其他（备注）

4. 执行失败

- 多步任务中断

- 状态丢失
- 重复执行
- 执行结果不可逆
- 其他（备注）

5. 用户体验失败

- 用户不知道发生了什么
- 用户无法纠正
- 用户无法撤回
- 用户对风险感知不足
- 其他（备注）

8.2 兜底机制

包括：

- 自动重试
- 降级回答
- 提示用户补充信息
- 转人工审核
- 限制AI权限
- 回滚操作
- 保存中间状态
- 失败原因解释
- 异常日志记录
- 监警告警
- 其他（备注）

优秀的AI Native产品不是永不失败，而是失败后不会让用户陷入不可控状态。

8.3 用户风险感知

重点判断：

- 用户是否知道AI即将执行什么？
 - 高风险操作前是否有确认？
 - 失败后用户是否知道如何补救？
 - 产品是否把风险转移给用户，而没有提供机制支持？
-

9. 商业模式与价值捕获

AI Native产品的商业模式往往不同于传统SaaS。

9.1 付费用户

- 个人用户
- 专业用户
- 团队
- 企业
- 开发者
- 平台生态伙伴

需要区分：

- 谁使用？
- 谁付费？
- 谁承担风险？
- 谁衡量ROI？

9.2 付费模式

常见模式：

- 订阅制
- 席位制
- 按Token计费
- 按任务次数计费
- 按工作流计费
- 按API调用计费
- 按结果或价值计费
- 企业定制部署收费
- 其他（备注）

9.3 AI对传统定价的冲击

分析：

- 是否从按席位收费转向按使用量收费？
- 是否在原SaaS基础上收取AI附加费？
- 是否因为模型成本导致毛利率承压？
- 是否把过去“人力服务”的价值产品化？

- 是否让低技能用户完成高技能任务，从而扩大市场？

9.4 ROI量化

重点看：

- 节省多少时间？
- 降低多少人力成本？
- 提高多少产出质量？
- 提升多少任务完成率？
- 降低多少错误率？
- 是否创造过去无法完成的新任务？

AI Native产品最强的商业价值，不只是“更快”，而是：**让用户用更低门槛完成更复杂的任务。**

10. 产品边界、战略与护城河

10.1 能力边界

明确区分：

- 哪些任务AI可以独立执行？
- 哪些任务必须人机协同？
- 哪些任务只能提供参考？
- 哪些任务绝对不能做？

尤其是：

- 法律
- 医疗
- 金融交易
- 高风险人事决策
- 安全相关操作
- 不可逆系统操作

10.2 竞品替代

分析：

- 它替代了哪类传统软件？
- 它替代了哪类人工服务？

- 它是否被大模型平台横向威胁？
- 它是否被原有SaaS厂商纵向集成威胁？
- 用户为什么不直接用通用大模型？

10.3 护城河

可能来源于：

1. **专有数据**
 - 行业数据、用户行为数据、任务反馈数据。
2. **工作流嵌入**
 - 产品深度嵌入用户日常流程，迁移成本高。
3. **上下文积累**
 - 产品越用越懂用户、团队和业务。
4. **工具生态**
 - 插件、API、第三方系统集成。
5. **可靠性工程**
 - 在高风险场景中更稳定、更可控。
6. **品牌信任**
 - 用户愿意让它处理重要任务。
7. **分发渠道**
 - 已有用户基础、企业客户、开发者生态。

10.4 扩展方向

分析未来可能：

- 从单点工具扩展为工作流平台
- 从Copilot升级为Agent
- 从文本扩展到多模态
- 从个人效率工具进入企业系统
- 从应用层下沉为基础设施
- 从通用助手转向垂直行业专家
- 其他（备注）

11. AI Native程度评分表

可以用以下表格对产品进行横向比较：

维度	评分问题	分数
任务重构	是否重塑核心任务链，而非只提升单点效率？	1-5
AI主路径	AI是否处于产品主流程，而非附加功能？	1-5
上下文深度	是否能读取、管理和利用多源上下文？	1-5
工具执行	AI是否能调用工具并完成真实操作？	1-5
可靠性机制	是否有校验、兜底、回滚和审计？	1-5
记忆能力	是否具备可控的短期和长期记忆？	1-5
商业重构	是否改变原行业定价、交付或价值捕获方式？	1-5
护城河	是否具备数据、 workflow、生态或信任壁垒？	1-5

判断标准

- 8-16分：普通AI功能型产品
 - 17-26分：AI增强型产品
 - 27-34分：准AI Native产品
 - 35-40分：高度AI Native产品
-